

Project budget control: one flow from request to commitment

Project X · next-generation multi-tenant ERP platform

AUTHOR	VERSION	STATUS	REVIEWERS
Hilal Sinem Sayar · Product Owner / Software Engineer	1.3 · 12 Aug 2026	Approved, phase 1 in flight	Finance · Projects · Eng Lead · CTO

Internal product document. Company names, tenant names and absolute figures have been anonymised or rounded for external sharing; research methods, ratios and product decisions are unchanged.

01 Summary

Every lira an engineering firm commits to a project passes through this domain. A site engineer raises a request, the request consumes project budget, the budget position decides which approval route it takes, and finance eventually reports on what was spent and against what.

On the legacy platform those four things lived in four places and agreed only by accident. The request form could not see the budget. The balance was recalculated inside a report, on a different screen, after the decision had been made. The approval route was fixed in code. Item names were free text, so category spend was a guess rather than a number. Each part worked; the seam between them was where the money went missing.

This release puts them in one flow. I own the domain across finance, projects and procurement, which in practice meant running the discovery and the market comparison, writing the spec, the acceptance criteria and the test cases, sitting in grooming, and writing a large part of the code. On top of the flow sits the platform's strategic bet: **natural-language record creation**, where a user describes what they need and the system proposes a filled form to review and submit. This domain is the right proving ground because a wrong suggestion here is cheap: the assistant never writes to the database.

OUTCOME THIS WORK COMMITS TO	BASELINE	TARGET
A committing decision is made with the budget in view	0% of requests showed a balance	100%, before submit
Spend is reportable by category without manual cleanup	31% of lines create a new material	≤ 3%
Entry is fast enough that people stop working around it	6 min 10 s to submit	≤ 3 min
AI-assisted entry proves itself on a low-risk flow	none yet	≥ 60% drafts accepted unedited

Offer and Purchase Order inherit the same budget contract over the following two quarters, so a data-model mistake made here is inherited twice. That is why section 04 records every decision with the alternative rejected.

02 Problem definition

The brief we were given was not a problem statement. It was "rebuild purchase requests on the new platform." Before writing a line of spec we ran two weeks of discovery to establish what was actually broken.

Quantitative analysis

Method. 18 months of production data across three tenant databases (≈11,400 requisitions, ≈47,000 lines). Material-name duplication measured by normalising each free-text item string (case-folding, Turkish-character folding, punctuation strip, token sort), then clustering by edit distance ≤ 2 on the normalised form. Cycle time measured submit → terminal approval, excluding never-submitted drafts.

FINDING	FIGURE	CONSEQUENCE
Lines that create a brand-new material record	31%	Legacy allowed ad-hoc item creation from the line itself
Of those, near-duplicates of an item that already existed	68%	"M8 bolt", "bolt M8", "M8 CIVATA" = three records, one item
Median cycle time, submit → final approval	3.1 days p90: 9.4 days	The tail, not the median, is what users remember
Requisitions returned for correction at least once	22%	Largest single contributor to the p90 tail

Requisitions submitted over the remaining project budget	17%	Not an error case but a legitimate separate approval route
Requisitions abandoned after being started	9%	Users open the form, hit friction, leave

TWO NUMBERS REFRAMED THE PROJECT

The **68% duplication rate** told us the material master is the real product problem, not the form. The **17% over-budget rate** told us that any design which *blocks* an over-budget requisition would break one in six real requests, a mistake our first solution sketch contained.

Qualitative research

Method. Nine semi-structured interviews (45 min) across three customer firms, plus four shadowing sessions watching real requisitions entered on site. Three site engineers, two purchasing specialists, two department managers, two finance controllers. Notes coded independently; themes counted by participants raising them unprompted.

THEME	RAISED BY	REPRESENTATIVE QUOTE
No budget visibility until rejection	8 / 9	"I find out there's no budget when someone rejects me three days later."
Catalogue search is slower than typing	7 / 9	"Searching takes longer than writing it, so I write it."
No idea where a request is sitting	7 / 9	"I call the purchasing guy to ask if it moved."
Approvers decide without context	5 / 9	"I open three screens to see if there's budget, then I approve anyway."
Reports don't tie	2 / 2 finance	"The same bolt has six names, so category spend is a guess."

Synthesis. Themes 2 and 5 are one problem seen from opposite ends: the catalogue is slow to search, so users bypass it, so finance cannot report. That causal chain drives the two largest decisions in this document. Themes 1, 3 and 4 are all *information at the wrong time*: the system already knows the answer, it just shows it after the decision instead of before.

Market comparison

Method. Three competing ERP products in the same segment, reviewed through their documentation, demo environments and release notes, plus a conversation with our two closest customers about what they used before and why they left it.

Two patterns came back, and both were openings rather than standards worth copying. Every product treats an over-budget request as an exception, blocked outright or released by a privileged user with an override, rather than as a route with its own approvers, which is exactly what our customers were reconstructing by phone. And in all three, budget is a report on a separate screen, consulted after the decision rather than during it. Neither is a technical limitation; both are old decisions nobody revisited.

PROBLEM STATEMENT

People commit money to a project without being able to see what is left of its budget, and they record what they are buying as free text because the catalogue is slower to search than to bypass. The first produces a 22% rework rate and a nine-day approval tail. The second leaves finance with spend data that cannot be reported by category without manual cleanup.

03 Goals, non-goals and success metrics

Goals

- **G1** · Make the catalogue the path of least resistance: entry through the material master must be *faster* than free text, not merely mandatory.
- **G2** · Move information before the decision: budget position, approval route and status visible at the moment they change the user's action.
- **G3** · Prove AI-assisted entry where a wrong suggestion is cheap.
- **G4** · Ship a budget contract Offer and Purchase Order can inherit without renegotiation.

Non-goals this release

- Purchase Order and Offer modules; this release only guarantees their contract
- FX revaluation and multi-currency reporting
- Native mobile app (responsive web only)
- Catalogue data *governance*: who curates material master data is a customer process question, not a product one

Success metrics

METRIC	TYPE	BASELINE	TARGET	SOURCE
Median time-to-submit	Primary	6 min 10 s	≤ 3 min	Event stream, form opened → submitted
New-material creation rate per line	Primary	31%	≤ 3%	Material inserts ÷ line count
Return-for-correction rate	Primary	22%	≤ 10%	Approval audit log

Median submit → final approval	Secondary	3.1 days	≤ 1.5 days	Approval audit log
p90 submit → final approval	Secondary	9.4 days	≤ 4 days	Approval audit log
AI draft accepted with no field edited	Leading	none yet	≥ 60%	Draft proposed vs. submitted events
Form abandonment	Counter-metric	9%	≤ 9%	Event stream

Guardrails: a breach stops the rollout

GUARDRAIL	THRESHOLD
Requisitions that reached a saved state while an active approval rule existed	0
Budget-ledger variance between amounts consumed and released	0.00
Database writes performed by the AI assistant	0, architecturally impossible (D6)
p95 form load	≤ 1.5 s

Users and jobs to be done

PERSONA	JOB TO BE DONE	CURRENT WORKAROUND	METRIC THEY MOVE
Site engineer ~70% of volume	"Get what I need ordered without leaving the site office."	Types free-text item names; phones purchasing for status	Time-to-submit, new-material rate
Purchasing specialist	"Turn a vague request into something I can actually buy."	De-duplicates and re-codes items after the fact	Return-for-correction rate
Dept. manager ~4 approvals/day	"Decide in under a minute without opening other screens."	Opens the budget report in a second tab, or approves blind	Cycle time, p90
Finance controller	"Report committed spend by category and project, accurately."	Excel cleanup before every report	New-material rate
Tenant admin	"Configure the approval flow to match our organisation."	Requests a code change from the vendor	Config-driven change ratio

04 Product decisions

Recorded with the alternatives rejected, because this is the part of the document the engineering team and I return to most often during grooming.

D1 • A material master table; lines cannot create items ad hoc

REJECTED Keep free-text entry and de-duplicate nightly with fuzzy matching. It makes the data *probably* right instead of right, and finance cannot audit a probabilistic merge.

TRADE-OFF Entry is slower unless search is genuinely fast. Mitigated by type-ahead over code, name and synonyms; a "recently used on this project" shortlist covering the repeat-order long tail; and an explicit *request a new material* action that routes to the catalogue owner instead of silently creating a record.

FALSIFIABLE New-material rate ≤ 3% **and** time-to-submit still ≤ 3 min. If the first improves while the second regresses, we made the catalogue mandatory rather than better, and D1 has failed on its own terms.

D2 • Budget is consumed at submit time, not at approval outcome

REJECTED Consume on approval. Rejected after modelling the race: two requisitions each within the remaining budget can both pass approval and jointly exceed it, and nothing in the system would have flagged it.

DETAIL The consumed amount **and its currency** are stored on the requisition, so the release on reject, withdraw or cancel refunds exactly what was taken even if the rate has moved. One live balance source; its remaining-amount column is computed and never written to.

D3 • Being over budget never blocks saving or submitting

Discovery said 17% of real requisitions are over budget. Out-of-budget is not an error state, it is a *different approval route*. The system derives a budgeted/not-budgeted flag after the lines are written and the approval engine selects the matching rule from it. The user is told, before submitting, which route they are entering.

D4 • Statuses live in a definition table, not in code

One status list is read by PRQ, and will be read by Offer, Purchase Order and reporting. No status code remains in the application layer: forward transitions come from approval-rule outcomes, rollback uses an "is initial" flag. **WHY IT IS A PRODUCT DECISION** a customer asking for one extra intermediate status becomes a configuration change instead of a release.

D5 • The approval engine owns "was this approved"; the module owns "what stage is it in"

Holding that line is what let us extend the engine so steps can hand structured data back to the module, including a fixed value applied automatically and hidden from the approver, which is how status transitions are driven, without any existing module's approval integration changing.

D6 • The AI assistant never saves

REJECTED Let the assistant create the record and notify the user. Rejected on two grounds: it creates a second write path bypassing the form's validation, and interview participants were uniformly uncomfortable with it: *"I want to see it before it goes."*

DESIGN The assistant returns the same payload the form already submits; the client holds it as a versioned, TTL-bounded draft and the form consumes it once. Unit price is deliberately absent from the assistant's schema, because a model should not be guessing money.

05 AI-assisted entry: the pilot

This is the part of the release that matters beyond PRQ. The assistant is meant to sit at the centre of the product, not beside it: the end state is a user asking for a report that does not exist as a screen and getting one. PRQ is where we find out whether that is real.

The flow

#	STEP	WHAT MAKES IT SAFE
1	Requester types in plain language: <i>"20 bags of cement and 5 rolls of insulation for the Block B site, needed next Wednesday"</i>	No schema is exposed to the model; it receives a form contract
2	Assistant resolves project, material group and materials through tools	Every tool call passes the same permission check the UI does; tools are off by default per tenant
3	A proposed draft lands in the form, every field visible and editable	Nothing is persisted; the draft expires on its own
4	Requester reviews, corrects, submits through the normal path	One validation path, one audit trail, one human accountable

Three findings that came out of testing, not planning

WHAT WE HIT	WHY IT HAPPENED	WHAT WE CHANGED
Common items needed three attempts to resolve	Material groups are stored in English; users speak Turkish. Text-searching the groups failed on the language boundary.	Return the full group list and let the model do the matching. Failure mode disappeared.
The assistant told a user their company had no requisitions	Row-level data scope filters <i>silently</i> . An empty result is indistinguishable from "none exist".	Empty results now carry an explicit note: state the scope limitation, never conclude absence.
The assistant re-listed rows the user could already see	We had written prohibitions into the tool description.	Reasons and examples outperform prohibitions. <i>"The user already sees the table"</i> worked where <i>"do not repeat the rows"</i> had not.

RULE ADOPTED FOR EVERY TOOL DESCRIPTION

Only state what is true in **every** tenant. Data rules, such as required fields and types, can be stated. Approval routing cannot, because it is configured per tenant. A tool description that is accurate for one customer and wrong for the next is worse than a vague one.

Decision rule, agreed with the CTO before the pilot ran

RESULT AFTER TWO WEEKS	ACTION
Acceptance $\geq$ 60% , cost per accepted draft within budget	Extend to a second module
Acceptance 35% to 60%	One iteration on tool descriptions and resolution, then re-measure
Acceptance $<$ 35%	Stop. The value is in the platform's tool layer, not the chat surface, and we say so plainly rather than keeping the feature alive because it demos well

Commercial and operational shape

Each tenant brings their own model-provider key: stored encrypted, only its last four characters ever returned to the client, with per-tenant usage and budget tracked: budget checked *before* each request, usage written after. The feature sits behind a per-tenant switch and is off

by default, so phase 3 rolls back without a deployment.

06 Interface

Purchase Requisition · New

SaveSubmit

Project Block B, MechanicalCompany (from session)

BUDGET · PROJECT BLOCK B

Allocated	Committed	Remaining	Used
1,200,000	1,043,000	157,000	87%

This requisition 64,200 → remaining after submit 92,800

LINES

cem

CEM-042Portland Cement 42.5R, 50 kg bag★ used on this project

CEM-019White Cement, 25 kg bag

+ Request a new material

1	CEM-042	Portland Cement 42.5R	20 bag	15 Sep	12,400
2	INS-113	Glass wool roll, 5 cm	5 roll	15 Sep	51,800

Requisition form. The budget strip answers theme 1 before the user acts rather than three days later; the search panel answers theme 2 by putting "used on this project" above everything else.

⚠ Over the remaining budget

RemainingThis requisition

157,000199,000

42,000 over the remaining project budget

Submitting routes this to Directorate approval instead of your department manager.

Submit anyway

Approval trail

Submitted

Department

Purchasing

Finance

12 Aug 09:14

Approved 11:02

With A. Demir since 2 h

Pending

Approval trail, answering theme 3. Each step names the actor and the elapsed time on the current step, which turns "where is it?" phone calls into a glance.

Over-budget state, 17% of real volume. Submission is permitted and the consequence is stated in words rather than left for the requester to infer.

07 Requirements

ID	USER STORY	ACCEPTANCE CRITERIA	PRI
R-01	As a requester I search the catalogue so I don't have to type item names	Given ≥3 characters typed, when results return, then matches on code, name or synonym appear within 300 ms (p95), and "recently used on this project" is shown before typing begins	P0
R-02	As a requester I request a new material without being blocked	Given no match, when I request a new material, then it is routed to the catalogue owner and my requisition stays editable as a draft	P0
R-03	As a requester I see my budget position before I submit	Given lines exist, when any quantity or price changes, then the strip reflects allocated / committed / remaining and this requisition's effect without a page reload	P0
R-04	As a requester I can submit over budget and know what that means	Given the total exceeds the remaining balance, when I submit, then submission succeeds and the routing consequence was stated on screen beforehand	P0
R-05	As finance I need consumption to be exact and reversible	Given a submitted requisition, when it is rejected, withdrawn or cancelled, then the released amount equals the stored consumed amount in the stored currency, regardless of later rate changes	P0
R-06	As a tenant admin I add an approval step without a release	Given a new rule and step, when saved, then new requisitions use it and in-flight requisitions keep the configuration copied at their submit time	P0
R-07	As an approver I decide without opening another screen	Given an assigned step, when I open it, then requester, project, budget position, lines and the over-budget flag are on one screen	P0

R-08	As a requester I see where my requisition is	Given a submitted requisition, when I open it, then each step shows state, actor and elapsed time on the current step	P0
R-09	As a requester I describe what I need in plain language	Given the assistant is enabled for my tenant and I hold the page permission, when I describe a need, then a fully editable draft is proposed and nothing is persisted until I submit	P1
R-10	As a tenant admin I bring my own model-provider key	Given a key, when saved, then it is stored encrypted, only the last four characters are ever returned, and per-tenant usage and budget are tracked	P1

Clarifications recorded during grooming, so they are not re-litigated: a requisition is always created under the logged-in user's company; there is no company selector; and approval routes through the *logged-in user's* company rather than the record's, which deliberately differs from legacy behaviour. Both were raised by engineers in grooming and confirmed correct with Purchasing.

08 Test cases

ID	SCENARIO	STEPS	EXPECTED
T-01	Happy path submit	Add two lines → save → submit	Advances to the first approval step; committed budget increases by the exact total
T-02	Over-budget submit	Add lines exceeding the balance → submit	Warning shown pre-submit; submit succeeds; flagged not-budgeted; routed to the over-budget rule
T-03	Reject refund after a rate change	Submit in currency A, change the rate, reject	Released amount equals the stored consumed amount and currency, not a re-converted figure
T-04	Concurrent submit on the last of the budget	Two drafts, each within the balance, jointly over; submit within the same second	Both submit; the second is flagged over-budget by the post-write balance; the ledger stays consistent
T-05	Approval-bypass attempt	Call the save endpoint directly, bypassing the UI	Server refuses a plain save on a page carrying an active submit rule
T-06	Permission gate on the rules endpoint	Open the form as a user lacking an unrelated page permission	Submit-for-approval action renders correctly; no silent fallback to a plain save
T-07	Unresolvable approver	Advance to a step naming someone who has left	Audit entry written; the request halts for reassignment and never auto-approves
T-08	Rule edited while requests are in flight	Edit the rule, then advance an open requisition	The in-flight request follows the configuration copied at its own submit time
T-09	Scope-filtered assistant query	Ask the assistant to list requisitions as a row-restricted user	Does not claim "no records exist"; states the scope limitation
T-10	Assistant draft is never persisted	Request a draft, close the browser without submitting	No row in any table; the draft expires with its TTL

09 Instrumentation and rollout

Events and the question each answers

EVENT	QUESTION
form opened (source)	Denominator for abandonment and time-to-submit
material search (query length, result count, selected rank, ms)	Is search faster than typing? Is the right item in the top three?
new material requested (had results?)	Are people requesting new items because nothing matched, or because they didn't look?
budget strip state	Live confirmation of the 17% over-budget figure
submitted (lines, total, budgeted, ms since open)	Primary metric
returned (step, reason code)	Rework driver analysis
draft proposed (cost, tools invoked, tools offered)	Cost per draft; whether tool crowding hurts selection

Phases and exit criteria

PHASE	EXIT CRITERIA
0 • Foundation internal	T-01...T-08 pass; ledger reconciles to 0.00 across a 500-requisition synthetic run
1 • Pilot tenant 1 tenant, ~25 users	Time-to-submit ≤ 4 min; new-material rate ≤ 8%; no guardrail breach for two weeks
2 • GA staged 10 → 50 → 100%	Primary metrics on trend at each gate
3 • Assistant pilot opt-in, tenant's own key	Acceptance ≥ 60%; cost per accepted draft within budget; zero writes

Rollback triggers. Any guardrail breach; return-for-correction rising above baseline for two consecutive weeks; p95 form load above 1.5 s.

10 Risks

#	RISK	L	I	MITIGATION
1	Catalogue quality is poor at a tenant, so mandatory selection blocks real work	H	H	"Request a new material" never blocks the draft; a catalogue health report is run per tenant before go-live
2	Users perceive the rebuild as slower than legacy	M	H	Time-to-submit is a shipping gate, not a post-launch metric; the pilot tenant is compared against its own legacy baseline
3	The assistant produces confidently wrong drafts	M	M	A human always submits; unit price excluded from the schema; acceptance measured per tenant
4	Provider cost per draft exceeds what the feature is worth	M	M	Per-tenant budget checked before each request, usage written after; the tenant uses their own key
5	Offer and PO later need a status this model cannot express	L	H	Statuses in a definition table (D4); a review with the Offer team is scheduled before their spec freezes
6	Migrated legacy requisitions distort the metric baseline	M	L	Migrated records are flagged and excluded from behavioural metrics

11 Stakeholders and ways of working

STAKEHOLDER	INTEREST	ROLE	CADENCE
Purchasing Lead	Data quality, buyability of requests	Accountable for catalogue rules	Weekly review
Finance Controller	Budget accuracy, reportability	Consulted on D2, D3, R-05	Sign-off per phase
Engineering Lead	Feasibility, model durability	Responsible for delivery	Daily standup, grooming
CTO	Platform strategy, AI direction	Consulted on D6 and the decision rule	Phase gates
Pilot tenant admin	Their own configuration	Informed , and the source of pilot feedback	Fortnightly call

How the team and I work. Two-week sprints, grooming on day two, where I walk the stories and the team pushes back. The clarifications in section 07 exist because two of those push-backs turned out to be right. Definition of Ready: acceptance criteria written, test cases drafted, no open question with a blocking owner. Definition of Done: criteria demonstrable on the pilot environment and the relevant events emitting.

WHERE CONSENSUS WAS WON THE HARD WAY

Finance wanted budget consumed on approval; Purchasing wanted it on submit. Two weeks of opinion-trading had settled nothing. I modelled the concurrency case with real numbers pulled from the legacy data (two requisitions, each individually within budget, jointly over) and showed that the approval-time model had no point at which either would be flagged. That ended the debate in one meeting.

12 Open questions

#	QUESTION	OWNER	NEEDED BY
1	Who owns catalogue curation at each tenant: is this a product feature or a customer process?	Purchasing Lead	Before phase 2
2	Should approval flows be editable as a visual tree rather than row by row?	Product Owner	Next quarter planning
3	How many resolution tools can the assistant carry before selection quality degrades? Instrumented, no answer yet.	Engineering	Before a second module adopts the assistant

Glossary. PRQ: purchase requisition · Tenant: a customer firm with its own database · Outcome: structured data an approval step hands back to the requesting module.